

UCS503 – Software Engineering
Mid-Semester Project Report

RiskTrace

Supply Chain Risk Analysis for
Open-Source Software Dependencies

Submitted by

1024030462 Shaurya Sangwan

1024030463 Abhishek Batra

1024030439 Sushain Sharma

Group: C32 Year: 2026

Programme: CSE

Faculty Advisor

Jeelani Sir

Mid-Semester Evaluation
Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Abstract

RiskTrace is a supply chain risk analysis tool for open-source software dependencies. It accepts a GitHub repository URL, discovers dependency manifests, extracts package information, checks dependencies against known vulnerability records, and presents a consolidated risk report with remediation guidance. The project addresses a practical development need: turning scattered dependency and advisory information into a reviewable, repository-level assessment.

At the mid-semester stage, the frontend and backend have been developed. The completed application pipeline covers repository registration, manifest discovery, dependency extraction, vulnerability matching, database persistence, and rule-based risk reporting. The frontend uses React, Vite, TypeScript, and Tailwind CSS; the backend uses Node.js with Prisma and PostgreSQL, with Docker configuration for the application environment. GitHub REST APIs supply repository information, while the OSV.dev API supplies vulnerability information.

Machine-learning-based risk prediction remains a planned extension, not a completed feature. The intended module will investigate package metadata and historical signals for prioritizing packages, including those with no known vulnerability match. This report documents the current system through original architecture and workflow diagrams, defines an evaluation strategy, and outlines the remaining development work. Quantitative performance and model accuracy remain to be evaluated.

Keywords: software supply chain, dependency analysis, GitHub API, OSV.dev, vulnerability prioritization, explainable machine learning.

Contents

Abstract	i
1 Introduction	1
1.1 Background and Context	1
1.2 Project Objectives	2
1.3 Scope and Current Progress	2
1.4 Project Team and Documentation	3
2 Methodology and Design	4
2.1 System Architecture	4
2.2 External API Integration	7
2.3 Implementation Details	8
2.4 Database Design	12
2.5 Use Cases and User Interaction	13
2.6 Planned AI/ML Risk Prediction	14
3 Results and Evaluation	16
3.1 Mid-Semester Results	16
3.2 Testing Methodology	17
3.3 Acceptance Scenarios	18
3.4 Performance Metrics	19
3.5 Analysis and Limitations	19
4 Conclusion	20
4.1 Summary of Work	20
4.2 Key Findings	20
4.3 Future Work and Recommendations	21
4.4 Final Remarks	21
Bibliography	22

List of Figures

2.1	RiskTrace architecture: current and planned components . . .	6
2.2	Scan lifecycle and error handling	9
2.3	Illustrative direct and transitive dependency graph	11
2.4	Database entities and relationships	13
2.5	Current user workflow and future ML features	14
3.1	Implemented stages and planned ML capabilities	17

List of Tables

1.1	Project objectives and milestone-based acceptance criteria. . .	2
1.2	RiskTrace project team.	3
2.1	Current technology stack and the planned analytics boundary.	5
3.1	Mid-semester implementation status, based on supplied project information.	16
3.2	Proposed acceptance scenarios; execution results have not yet been supplied.	18
3.3	Evaluation metrics without fabricated benchmark results. . . .	19

CHAPTER 1

Introduction

1.1 Background and Context

RiskTrace focuses on the dependencies that an application obtains from open-source package ecosystems. A repository can declare a relatively small set of direct dependencies while relying on additional packages through those dependencies. The project therefore treats dependency visibility and vulnerability interpretation as connected tasks: identifying package names alone is not sufficient for a useful risk report.

The intended users are developers, student project teams, and maintainers who need a practical starting point for dependency review. Rather than requiring users to inspect manifests and advisory records separately, RiskTrace brings repository collection, package analysis, and reporting into one workflow. The project website describes an implemented repository-to-report pipeline and identifies AI/ML analysis as upcoming [Ris].

1.1.1 Problem Statement

The problem addressed by RiskTrace is the lack of a single, accessible workflow for answering four questions about a repository:

1. Which open-source dependencies and versions can be identified?
2. Which identified versions have known vulnerability matches?
3. Which findings should receive attention first?
4. What remediation information is available for those findings?

The system must also communicate what it could not determine. A missing version, unsupported manifest, unavailable external service, or incomplete dependency graph should not be interpreted as evidence that the repository is safe.

1.2 Project Objectives

Table 1.1 separates the mid-semester deliverables from the next development phase. Completion status follows the team-provided project summary; the acceptance criteria define how the work should be demonstrated, rather than asserting that a formal test suite has passed.

Objective	Acceptance criterion	Milestone / status
Repository intake	A valid URL produces repository metadata and a distinct scan record.	Mid-semester: implemented.
Dependency inventory	Supported manifests yield package, ecosystem, version, and dependency-type information where resolvable.	Mid-semester: implemented; coverage requires fixture-based verification.
Vulnerability reporting	Matched records retain identifiers, available severity information, and available fixed versions.	Mid-semester: implemented.
Risk dashboard	A completed scan exposes a score, risk category, findings, and remediation guidance.	Mid-semester: frontend and backend developed.
Predictive extension	Train and evaluate a package-risk baseline on a documented, leakage-controlled dataset.	Next phase: not implemented.

Table 1.1: Project objectives and milestone-based acceptance criteria.

These objectives are specific and measurable through observable outputs. Their time boundaries are the mid-semester review and the subsequent evaluation milestone; calendar deadlines should be confirmed with the faculty advisor. The next phase should prioritize evaluation and a minimal predictive baseline before adding complex model explanations.

1.3 Scope and Current Progress

The current scope is repository-level dependency analysis and known-vulnerability reporting. The completed frontend provides the user-facing workflow, while the backend handles repository collection, analysis, persistence, and report generation. Risk categories are **LOW**, **MEDIUM**, **HIGH**, and **CRITICAL**.

The project summary lists `package.json`, `requirements.txt`, `pom.xml`, and `Gemfile` as manifest examples. These examples define the intended multi-ecosystem scope; they do not establish that every ecosystem’s version syntax or transitive resolution case has been validated. The evaluation chapter includes those checks.

ML predictions, SHAP explanations, and package-health enrichment belong to the future scope. RiskTrace does not currently claim to identify undisclosed vulnerabilities, prove exploitability, execute malware analysis, or guarantee that an unmatched package is secure.

1.4 Project Team and Documentation

Team member	Roll number
Shaurya Sangwan	1024030462
Abhishek Batra	1024030463
Sushain Sharma	1024030439

Table 1.2: RiskTrace project team.

CHAPTER 2

Methodology and Design

2.1 System Architecture

2.1.1 Current Implementation

RiskTrace follows a repository-to-report pipeline:

1. **Repository input:** accept the GitHub URL, identify the owner and repository, fetch metadata, and register a scan.
2. **Manifest discovery:** examine the repository tree and locate supported dependency manifests.
3. **Dependency extraction:** parse available package and version information and associate the resulting records with the scan.
4. **Vulnerability matching:** query OSV for identified package versions and retain relevant advisory information.
5. **Risk reporting:** aggregate vulnerability information into the current rule-based score and display findings and recommendations.

The separation between extraction, matching, and reporting is the intended integration boundary for the later ML module. A prediction service should enrich a successful scan without becoming a prerequisite for known-vulnerability reporting.

2.1.2 Technical Stack

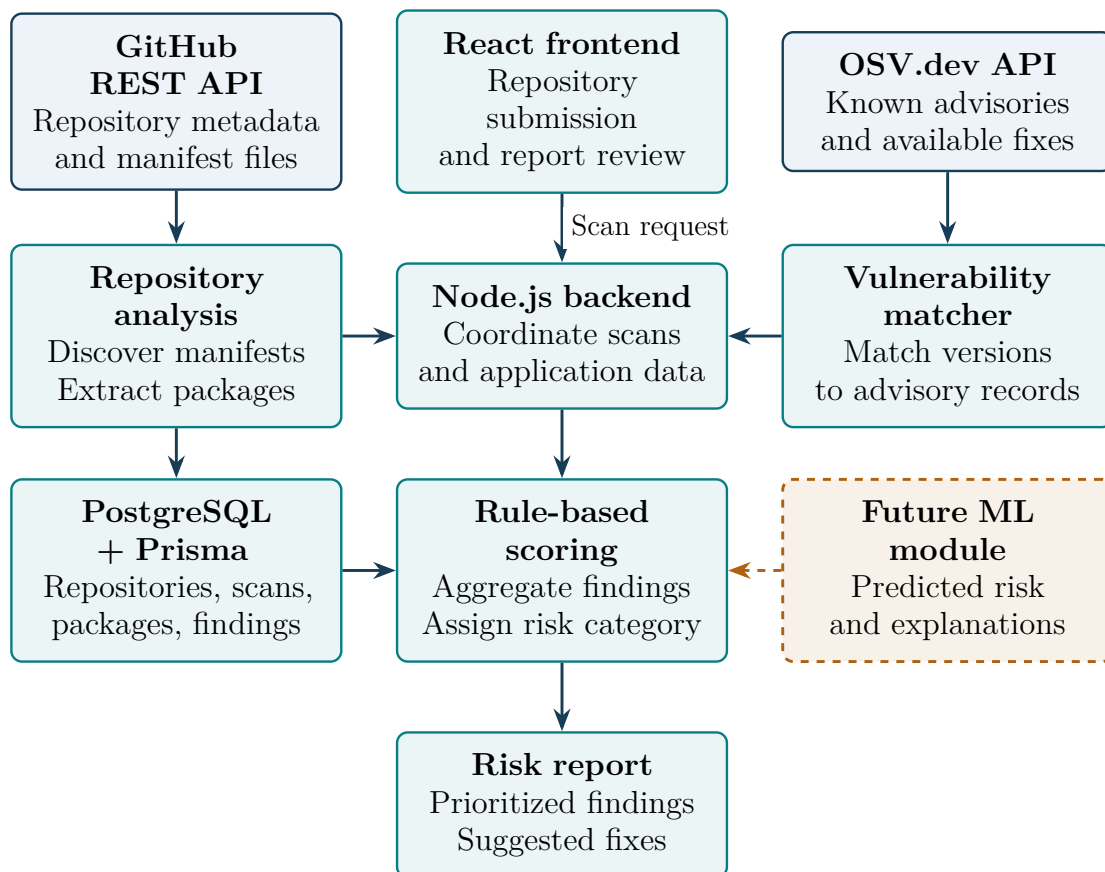
Layer	Technology	Role in RiskTrace
Frontend	React, Vite, TypeScript, Tailwind CSS	Repository input and presentation of scan outcomes and dependency findings.
Backend	Node.js	Scan coordination, external API integration, and risk-report assembly.
Persistence	Prisma, PostgreSQL	Repository, scan, dependency, and vulnerability storage.
Infrastructure	Docker, Docker Compose	Configured application and database environment.
External services	GitHub REST API, OSV.dev API	Repository retrieval and known-vulnerability lookup.
Future analytics	Candidate Python ML service	Feature preparation, inference, and optional explanations; not implemented.

Table 2.1: Current technology stack and the planned analytics boundary.

2.1.3 Implemented Architecture and Extension Boundary

Figure 2.1 redraws the architecture around the current Node.js backend. It separates repository analysis, advisory matching, persistence, scoring, and report presentation. The frontend and backend are developed; the dashed ML component represents the next implementation phase.

The original design explored a Python/FastAPI service and additional package-health sources. These remain optional directions for the prediction module, not replacements for the completed backend. The present architecture uses GitHub repository APIs and OSV.dev for its external data inputs.



Solid teal: current application **Dashed amber: planned extension**

Arrows summarize information flow, not individual API calls.

Figure 2.1: RiskTrace system architecture, redrawn for the current implementation. Solid teal components belong to the existing application; the dashed amber module is planned.

2.2 External API Integration

2.2.1 GitHub REST API

Repository discovery uses the GitHub REST API. The base address is `api.github.com`; the relevant read operations are listed below [Gitb; Gita; Gitc].

```
GET https://api.github.com/repos/{owner}/{repo}
GET https://api.github.com/repos/{owner}/{repo}/git/trees/{tree_sha}?
    recursive=1
GET https://api.github.com/repos/{owner}/{repo}/contents/{path}?ref={ref}
```

Repository metadata supplies the default branch. Tree retrieval supports manifest discovery, while the contents endpoint retrieves selected files. GitHub can mark a recursive tree response as truncated; a complete scan must traverse the missing subtrees or explicitly report incomplete discovery [Gita].

For reproducible scans, the recommended refinement is to resolve the selected branch to a commit and fetch manifests against that revision. Input validation should accept supported GitHub repository URLs rather than arbitrary fetch targets. Credentials, where required, should remain on the backend and never appear in frontend code or reports. These are hardening requirements, not assertions that all safeguards have already been audited.

2.2.2 OSV.dev Vulnerability API

The vulnerability service is **OSV.dev** (Open Source Vulnerabilities). Its package-version query endpoint accepts an ecosystem, package name, and version [OSVb]:

```
POST https://api.osv.dev/v1/query
Content-Type: application/json

{
  "package": {"name": "jinja2", "ecosystem": "PyPI"},
  "version": "2.4.1"
}
```

This is an API-format example, not a RiskTrace benchmark or a recommendation to install that version. OSV records can contain advisory identifiers, aliases, affected versions, severity data, and fixed-version events. A CVE alias,

numeric CVSS score, or fixed version is not guaranteed for every record [Opeb]. The matcher should preserve missing values as unknown rather than convert them to zero severity. It should also distinguish a successful lookup with no matches from a failed lookup. Version constraints that cannot be resolved should be flagged rather than sent as if they were exact installed versions.

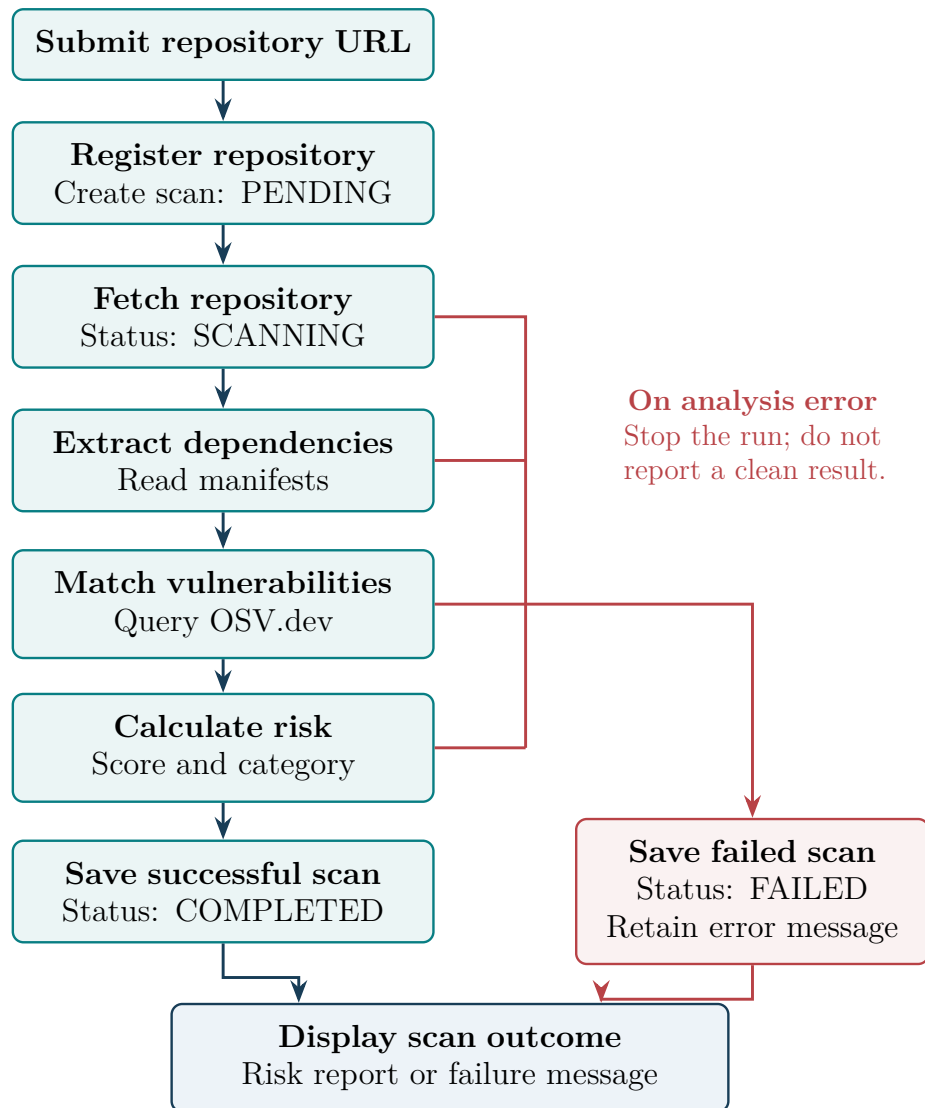
For later optimization, OSV provides `POST /v1/querybatch`. Batch responses contain identifiers and modification timestamps rather than complete vulnerability records; full details can be retrieved using `GET /v1/vulns/{id}`. Pagination must also be handled [OSVc; OSVa]. Batch processing is a proposed enhancement here, not a confirmed current implementation detail.

2.3 Implementation Details

2.3.1 Module 1: Repository Intake and Scan Lifecycle

The backend registers repository information and creates a separate scan record for each analysis run. The activity and database diagrams use the states `PENDING`, `SCANNING`, `COMPLETED`, and `FAILED`. A successful run stores the report outcome; a failed run stores an error message rather than presenting a misleading successful result.

The lifecycle in Figure 2.2 makes error handling part of the workflow. Repository access, manifest parsing, vulnerability retrieval, and scoring can each prevent completion. A useful failure response should identify the failing stage, retain a scan identifier, and avoid leaking credentials or internal connection strings. The frontend should present a report or a failure outcome, not leave an unsuccessful scan looking like a clean result.



The red branch merges alternative failure paths; it is not parallel execution.

Figure 2.2: Redrawn scan activity flow. Successful stages follow the blue path; an analysis error follows the red branch to a failed outcome.

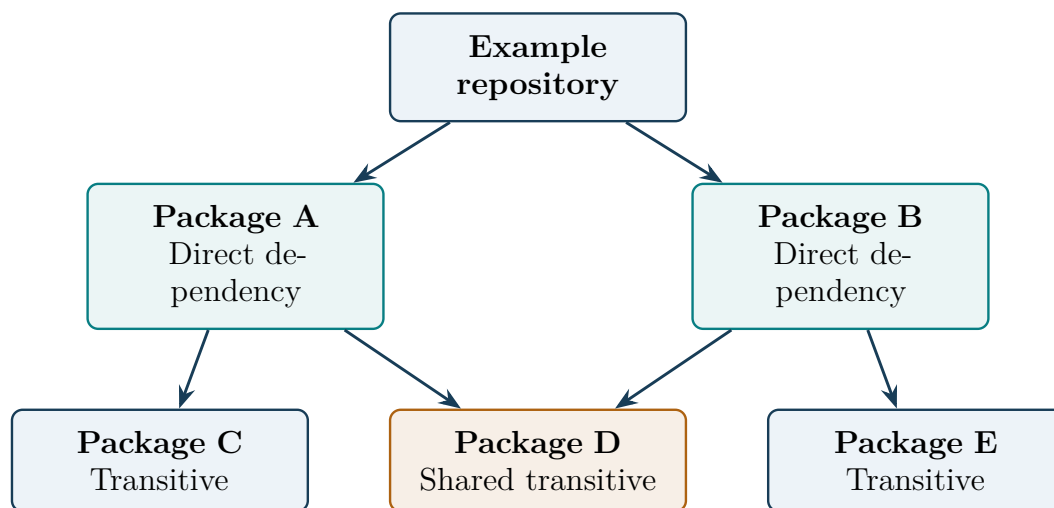
2.3.2 Module 2: Manifest Discovery and Dependency Extraction

Manifest discovery identifies files by supported ecosystem conventions. Extraction converts their contents into dependency records containing a name, ecosystem, version, and direct/transitive indicator. A repository with several manifests may produce dependencies from several subprojects; identical names from different ecosystems must remain distinct.

The implementation summary reports direct and transitive extraction. The important validation boundary is how exact versions and indirect relationships are obtained. Manifest-only declarations do not establish a complete resolved graph in every case. Acceptance tests should distinguish exact pinned versions, unresolved ranges, and dependencies recovered from lockfiles or resolver data. Where such evidence is unavailable, the report should disclose the gap rather than invent a transitive closure.

For future traceability, each item should retain its manifest path and resolution source. This provenance is an extension to the supplied schema, not an existing field shown in the entity–relationship diagram. A similar extension could store dependency edges to display the parent path leading to a transitive vulnerability.

Figure 2.3 illustrates why dependency identity and dependency paths must be handled separately. The package labels are fictional: this graph explains the analysis concept and is not an extracted RiskTrace scan.



Two dependency paths reach Package D. A package-level inventory should distinguish dependency paths from unique package/version identities.

Figure 2.3: Illustrative dependency graph with two direct dependencies and three transitive packages. Package D is reachable through two parents; amber highlights a shared dependency, not a vulnerability or measured risk.

2.3.3 Module 3: Vulnerability Matching and Recommendations

The matching stage associates dependency records with known advisories. Its output supplies package-level findings for the dashboard and vulnerability data for the aggregate score. The recommendation layer uses available fixed-version information to suggest upgrade review, while retaining the original advisory identifier.

Recommended reporting rules are to group advisory aliases so one issue is not counted several times for the same package version, preserve separate findings for distinct affected dependencies, and avoid describing an upgrade as universally safe. Compatibility testing remains necessary. If a transitive package is affected, remediation may require updating a direct parent rather than editing an indirect package declaration.

2.3.4 Module 4: Current Risk Scoring and Dashboard

The current score is a rule-based aggregation of vulnerability information. The project describes using severity, CVSS, and finding counts, followed by assignment to one of four risk categories. The exact formula, weights, and category thresholds were not supplied; this report therefore does not invent

them or present an illustrative formula as production behavior.

Before quantitative evaluation, the scoring configuration should be recorded alongside the implementation revision. Acceptance properties should include deterministic output for identical inputs, a defined policy for missing severity, no double-counting of advisory aliases, and a distinction between “no known matches” and “not assessed.” A high-severity finding should remain visible even in a large repository with many unmatched packages.

The frontend’s purpose is to make the result actionable: identify the repository, show the aggregate assessment, present affected packages and available fixes, and allow users to inspect the evidence. Additional recommended fields are scan time, assessed revision, coverage limitations, and separate labels for future predictions. A risk category is a prioritization aid, not a measured probability of exploitation.

2.4 Database Design

Figure 2.4 describes four main entities:

Repository

Stores owner, name, unique URL, default branch, optional latest commit SHA, and timestamps.

Scan

References a repository and stores lifecycle state, timing, optional score and risk level, error text, ecosystems, discovered manifests, and file count.

Dependency

References a scan and stores package name, version, ecosystem, and the direct-dependency indicator.

Vulnerability

References a dependency and stores the advisory identifier, severity, optional CVSS value, summary, and fixed versions.

The relationships are one repository to many scans, one scan to many dependencies, and one dependency to many vulnerability findings. The diagram specifies cascading deletion for the latter two relationships, but not for repository-to-scan deletion. Scan-specific dependency records retain the findings of individual runs instead of treating a repository’s dependency list as timeless.

Two schema refinements merit consideration: storing the assessed commit

on each scan rather than only the repository's latest commit, and retaining optional provenance/coverage information for dependencies. Both refinements remain future work and are not included as implemented fields in the database diagram.

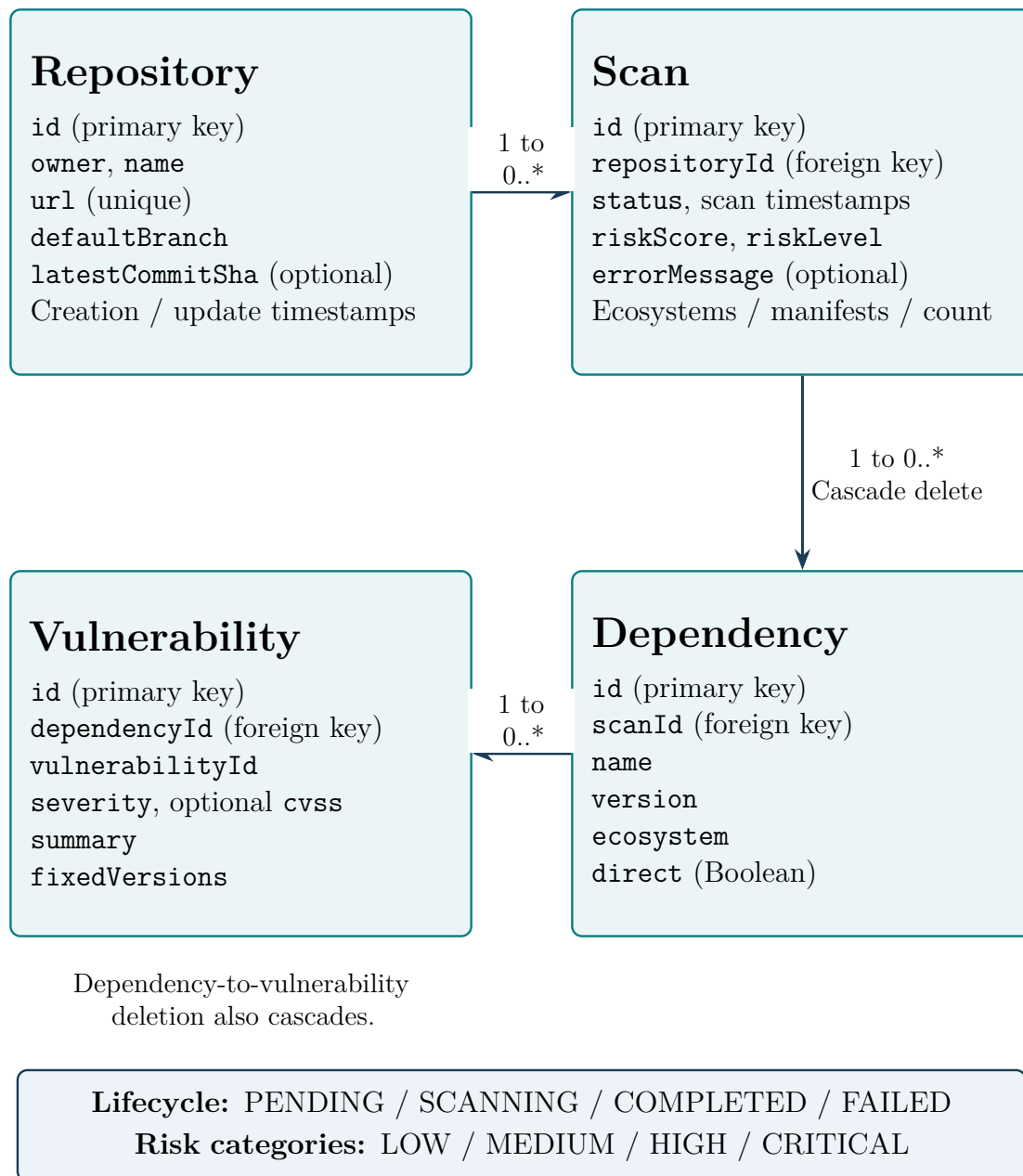


Figure 2.4: Redrawn entity–relationship view of the Prisma/PostgreSQL data model. Each parent can have zero or more child records; key fields are shown for readability.

2.5 Use Cases and User Interaction

The primary user submits a repository, reviews a completed assessment, inspects dependency findings, and reruns a scan after changes. Figure 2.5

distinguishes these current interactions from future prediction and model-explanation capabilities. The existing dashboard reports advisory-backed findings; predicted package risk will be a separate, explicitly labelled output.

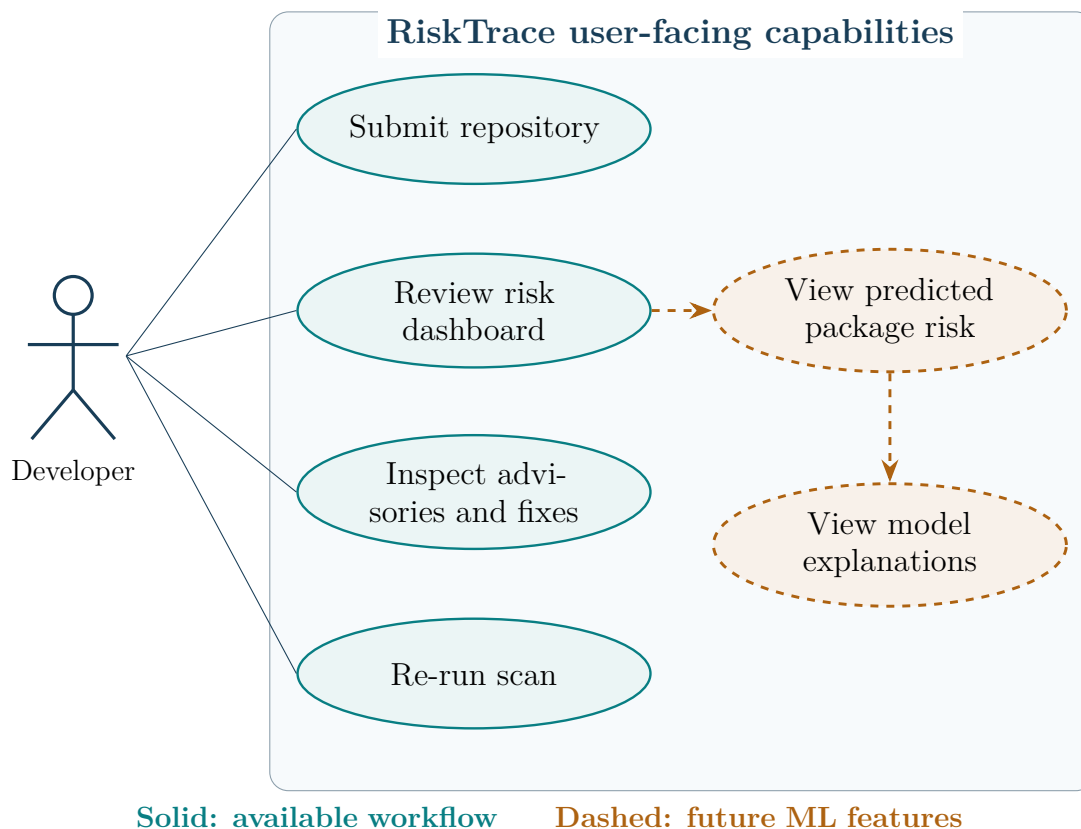


Figure 2.5: Redrawn user-capability diagram. Current repository analysis and report inspection are distinguished from future predictive and explanation features.

2.6 Planned AI/ML Risk Prediction

2.6.1 Purpose and Candidate Inputs

The next phase will investigate whether package metadata and historical vulnerability patterns can improve review prioritization, including for packages with no current OSV match. Such output should be labelled *predicted risk* and kept separate from confirmed advisory matches. A prediction must not be presented as evidence that an undisclosed vulnerability exists.

Candidate features include historical advisory counts, historical severity summaries, release recency, maintenance activity, and dependency-graph properties where reliable data are available. deps.dev provides package dependency information and is a possible enrichment source [Opea]. Its use, along with candidate Scorecard-derived features, remains planned.

2.6.2 Datasets and Label Definition

Candidate datasets are OSS Vulnerabilities on Kaggle and Datadog’s malicious-software-packages collection [jap; Dat]. Dataset snapshots, preprocessing, and training remain to be completed.

Disclosed vulnerabilities and intentionally malicious packages are different targets and should not be merged into one label. Datadog’s collection may support a separate malicious-package experiment. Absence of an advisory is not a verified negative label.

The proposed first experiment predicts new advisory disclosure within a defined future window, using only information available at the prediction cutoff. Historical snapshots and timestamps must support this task. Learning labels calculated from current CVSS and advisory counts would reproduce a ranking rule, not demonstrate prediction of unseen vulnerabilities.

2.6.3 Training, Evaluation, and Integration Plan

1. Define the target, observation window, dataset snapshot, and inclusion criteria.
2. Normalize package identities, remove duplicates, and document missing values and class balance.
3. Construct chronological evaluation splits and audit package overlap to reduce leakage.
4. Compare a simple heuristic and logistic-regression baseline with candidate tree-based models such as Random Forest or XGBoost.
5. Evaluate precision, recall, F1, precision–recall area, and precision among the highest-ranked candidates; evaluate calibration if producing probability estimates.
6. Integrate a versioned model behind an optional backend interface, retaining the existing reporting workflow when inference is unavailable.

SHAP is a candidate explanation mechanism for the planned ML module, not an implemented capability. Explanations should describe model contributions rather than imply causal proof. Final model selection and thresholds will depend on measured results.

CHAPTER 3

Results and Evaluation

3.1 Mid-Semester Results

The achieved milestone is the development of the frontend and backend for the known-vulnerability analysis workflow. The deliverables reported by the team cover repository intake through risk reporting, with persistence through Prisma and PostgreSQL. The remaining major functional component is the ML prediction module.

Deliverable	Current status	Evidence / evaluation boundary
Frontend interface	Developed	Team-reported repository input and report workflow.
Backend scan pipeline	Developed	Team-reported intake, discovery, extraction, matching, and scoring.
Database integration	Implemented	Prisma/PostgreSQL stack and supplied entity relationships.
GitHub and OSV integration	Implemented	Reported repository retrieval and vulnerability lookup.
Container configuration	Available	Docker Compose configuration reported; load validation remains to be documented.
System documentation	Available	Proposal, pitch, and supplied system diagrams.
ML training and inference	Not implemented	Dataset preparation, training, and serving remain future work.
SHAP explanations	Not implemented	Included only in the target design.

Table 3.1: Mid-semester implementation status, based on supplied project information.

Implementation status records the team’s development milestone. Quantitative benchmarks and formal acceptance-test results are not yet documented; the following protocol defines the evidence needed for the next review.

3.1.1 Implementation Progress at a Glance

Figure 3.1 counts the five implemented pipeline stages listed in Section 2.1.1: repository intake, manifest discovery, dependency extraction, vulnerability matching, and risk reporting. It contrasts them with two planned ML capabilities: prediction and model explanations. This is a count of listed functions, not a percentage of project completion, an effort estimate, or a performance benchmark.

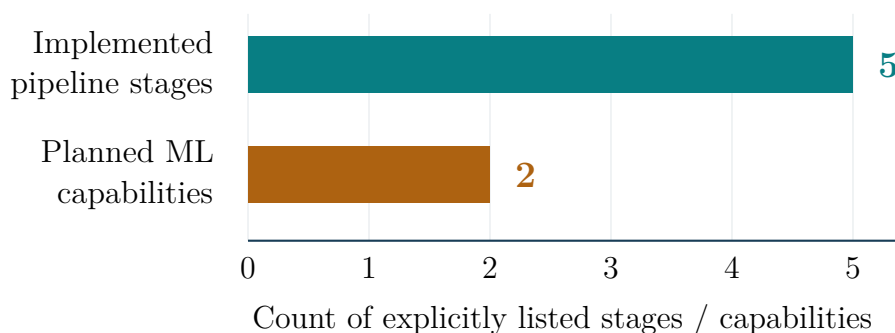


Figure 3.1: Functional-scope counts derived from the project status: five implemented pipeline stages and two planned ML capabilities.

3.2 Testing Methodology

3.2.1 Unit Testing

Parser fixtures should cover exact versions, ranges, malformed files, empty manifests, duplicate declarations, ecosystem-specific naming, and available lockfile information. Vulnerability normalization tests should cover no-match responses, missing severity, multiple advisory aliases, and absent fixed versions. Scoring tests should use frozen findings and the documented production configuration.

3.2.2 Integration Testing

A controlled repository should exercise the entire path from URL submission to persisted report. Mocked GitHub and OSV responses should make failure scenarios reproducible without depending on live services. Separate live smoke tests should verify the external contracts. Database checks should verify foreign-key associations, scan isolation, and finalization on failure.

3.2.3 Performance and User Testing

Performance evaluation should measure complete scan duration rather than compare the pipeline with a generic single-request target. Repositories should be grouped by manifest and dependency counts, with repeated runs distinguishing cold from cached execution. Suggested measurements include median and 95th-percentile scan time, external request count, and peak resource usage. These are proposed measurements, not achieved values.

For usability evaluation, participants should submit a repository, identify its highest-priority finding, locate a suggested fix, and interpret a failed scan. Record task completion and misunderstandings, especially whether users mistake “no known vulnerabilities” for a guarantee of safety. No participant study is claimed at this stage.

3.3 Acceptance Scenarios

Input / condition	Expected outcome to verify
Accessible repository with a pinned manifest	Scan completes and package versions are associated with the correct scan.
Controlled affected package/version	Expected advisory identifiers and available remediation fields appear.
Successful lookup with no matches	Report states that no known matches were returned, without claiming proven safety.
Unresolved version or incomplete dependency data	Coverage limitation is visible rather than represented as a clean exact-version result.
Invalid or inaccessible repository	Clear input/access error, without a misleading successful assessment.
GitHub or OSV failure	Failed or explicitly incomplete outcome; no fabricated zero-risk result.
Repeated scan of unchanged fixtures	Separate scan identity with reproducible findings and score.
Unavailable future ML module	Current known-vulnerability pipeline remains usable without predictions.

Table 3.2: Proposed acceptance scenarios; execution results have not yet been supplied.

3.4 Performance Metrics

Metric	Measurement / target definition	Result available
Extraction correctness	Agreement with a manually checked inventory; all supported fixture expectations should pass.	Not measured.
Advisory-match correctness	Expected versus returned advisory sets for pinned package/version fixtures.	Not measured.
End-to-end latency	Median and 95th percentile over repeated scans, grouped by repository size.	Not measured.
Failure handling	Each injected failure yields the documented outcome and persisted status.	Not measured.
Score reproducibility	Identical frozen findings and configuration yield identical scores.	Not measured.
Predictive performance	Held-out precision, recall, F1, ranking quality, and applicable calibration metrics.	Not applicable yet; ML not implemented.

Table 3.3: Evaluation metrics without fabricated benchmark results.

3.5 Analysis and Limitations

The completed pipeline provides a foundation for predictive analysis. Its separation of evidence collection and scoring allows future models to be evaluated without replacing known-vulnerability reporting. The remaining validation priorities are:

- **Coverage:** verify exact-version and transitive resolution for each supported ecosystem.
- **Evidence quality:** disclose unknown versions, missing severity, and unsuccessful external lookups.
- **Scoring:** document thresholds and missing-data policies before comparing results.
- **Reliability:** make access failures and incomplete discovery visible; keep prediction optional.
- **Evaluation:** record correctness tests, benchmarks, usability observations, and future model results.

CHAPTER 4

Conclusion

4.1 Summary of Work

RiskTrace has reached the mid-semester milestone with its frontend and backend developed for repository-based dependency risk analysis. The reported pipeline connects GitHub repository collection, manifest discovery, dependency extraction, OSV vulnerability matching, persistent storage, and rule-based reporting. The project has a functional foundation on which to build the proposed predictive extension.

The current system brings repository collection, dependency records, advisory evidence, and developer-facing reporting into a single workflow. The next milestone is to validate that workflow systematically and integrate a carefully evaluated predictive extension.

4.2 Key Findings

- **The current milestone is application integration.** The frontend, backend, and persistence pipeline are delivered components; ML is not yet part of the completed system.
- **Risk reporting needs traceable evidence.** Package versions, advisory identifiers, scan context, and missing-data handling determine whether users can interpret a report responsibly.
- **Predictive risk needs an independent evaluation.** A future model must be assessed against a clearly defined target rather than credited with detecting unknown vulnerabilities merely because it produces a score.

These are development conclusions and design lessons, not statistical findings from an experiment.

4.3 Future Work and Recommendations

1. **Validate the existing pipeline:** execute the acceptance matrix, publish fixture-based results, and document the scoring formula and category thresholds.
2. **Strengthen traceability:** expand verified ecosystem coverage, retain manifest provenance, and record the exact revision assessed by each scan.
3. **Prepare the ML dataset:** select a prediction target, record dataset snapshots, audit labels, and construct temporal evaluation splits.
4. **Implement a baseline predictor:** compare baseline and candidate models and report held-out results, including failure modes and class imbalance.
5. **Integrate prediction cautiously:** add a versioned optional inference interface, separate predictions from advisory-backed findings, and add explanations after validation.
6. **Improve operational readiness:** evaluate bounded concurrency, caching, batch lookups, background execution, credential handling, and container deployment under controlled load.
7. **Extend developer workflows:** consider scan-history comparisons, dependency-path views, CI integration, and exportable reports after core correctness is demonstrated.

4.4 Final Remarks

The immediate priority is to demonstrate that the completed scan pipeline is accurate, reproducible, and understandable. ML is the next major implementation task, but its value will depend on careful data preparation and honest evaluation. By maintaining a clear boundary between known vulnerabilities, unavailable evidence, and future predictions, RiskTrace can develop into a useful dependency-review tool without overstating what an automated assessment can establish.

Bibliography

- [Dat] Datadog. *Malicious Software Packages Dataset*. Candidate research input; no training results claimed. URL: <https://github.com/DataDog/malicious-software-packages-dataset> (visited on 09/13/2026) (cit. on p. 15).
- [Gita] GitHub. *REST API Endpoints for Git Trees*. URL: <https://docs.github.com/en/rest/git/trees> (visited on 09/13/2026) (cit. on p. 7).
- [Gitb] GitHub. *REST API Endpoints for Repositories*. URL: <https://docs.github.com/en/rest/repos/repos> (visited on 09/13/2026) (cit. on p. 7).
- [Gitc] GitHub. *REST API Endpoints for Repository Contents*. URL: <https://docs.github.com/en/rest/repos/contents> (visited on 09/13/2026) (cit. on p. 7).
- [jap] japkeeratsingh. *OSS Vulnerabilities*. Dataset identified by the project; experimental snapshot not specified. Kaggle. URL: <https://www.kaggle.com/datasets/japkeeratsingh/oss-vulnerabilities/data> (cit. on p. 15).
- [Opea] Open Source Insights. *deps.dev Documentation*. URL: <https://docs.deps.dev/> (visited on 09/13/2026) (cit. on p. 14).
- [Opeb] Open Source Security Foundation. *Open Source Vulnerability Format*. URL: <https://ossf.github.io/osv-schema/> (visited on 09/13/2026) (cit. on p. 8).
- [OSVa] OSV. *OSV API Documentation*. URL: <https://google.github.io/osv.dev/api/> (visited on 09/13/2026) (cit. on p. 8).
- [OSVb] OSV. *POST /v1/query: Query for Vulnerabilities*. URL: <https://google.github.io/osv.dev/post-v1-query/> (visited on 09/13/2026) (cit. on p. 7).
- [OSVc] OSV. *POST /v1/querybatch: Query for Multiple Packages*. URL: <https://google.github.io/osv.dev/post-v1-querybatch/> (visited on 09/13/2026) (cit. on p. 8).
- [Ris] RiskTrace Team. *RiskTrace: Supply Chain Risk Analysis*. URL: <https://aishaurya777.github.io/RiskTrace/> (visited on 09/13/2026) (cit. on p. 1).